Apunte: Introducción a la Programación - Guía 2

Por Marto
Facultad de Ciencias Exactas y Naturales
UBA

Última actualización: 2 de abril de 2026

The Council Will Decide Your Fate:

(doble click en los ejercicios para saltar)

- Ejercicio 1
- Ejercicio 2
- Ejercicio 3
- Ejercicio 4
- Ejercicio 5
- Ejercicio 6
- Ejercicio 7
- Ejercicio 8

¡Recomendación para sacarle jugo al apunte!

Disclaimer: Recomendable leer lo siguiente, ayuda banda si no estas preparado para encarar la guía usando el repo.

Estudiar con resueltos puede ser un arma de doble filo. Si estás trabado, antes de saltar a la solución que hizo otra persona:

1. **No mires la solución ni bien te trabás.** Te condicionás pavlovianamente a no pensar. Necesitás darle tiempo al cerebro.
2. Intentá un ejercicio similar, pero más fácil. ¿No sale? Intentá uno aún más fácil.
3. Fijate si tenés un ejercicio similar hecho en clase.
4. Tomate 2 minutos para formular una pregunta real sobre lo que no entendés.

Eh loco, relajá un toque... ¡va a salir todo bien!

2. Especificación de problemas

Ejercicio 1. Dadas las siguientes especificaciones, dar valores de entrada y salida que cumplan con los requiere y asegura respectivamente:

a) **problema** duplicar (x: $\mathbb{Z}$): $\mathbb{Z}$ {
    requiere: { True }
    asegura: { resultado es el doble de x }
}

b) **problema** raizCuadrada (x: $\mathbb{Z}$): $\mathbb{R}$ {
    requiere: { x es positivo }
    asegura: { resultado es la raíz cuadrada de x }
}

c) **problema** enteroMasCercanoPositivo (x: $\mathbb{R}$): $\mathbb{Z}$ {
    requiere: { True }
    asegura: { resultado es el entero más cercano de x }
    asegura: { resultado es positivo }
}

d) **problema** raicesCuadradasUno (s: $seq\langle\mathbb{Z}\rangle$): $seq\langle\mathbb{R}\rangle$ {
    requiere: { Todos los elementos de s son positivos }
    requiere: { No hay elementos repetidos en s }
    asegura: { resultado tiene la misma cantidad de elementos que s }
    asegura: { Los elementos de resultado son la salida de aplicar el problema raizCuadrada a todos los elementos de la secuencia s }
    asegura: { El orden de la secuencia resultado es el mismo que en la secuencia s, luego de aplicar el problema raizCuadrada }
}

e) **problema** raicesCuadradasDos (s: $seq\langle\mathbb{Z}\rangle$): $seq\langle\mathbb{R}\rangle$ {
    requiere: { Todos los elementos de s son positivos }
    requiere: { No hay elementos repetidos en s }
    asegura: { resultado tiene la misma cantidad de elementos que s }
    asegura: { Los elementos de resultado son la salida de aplicar el problema raizCuadrada a todos los elementos de la secuencia s }
}

f) **problema** raicesCuadradasTres (s: $seq\langle\mathbb{Z}\rangle$): $seq\langle\mathbb{R}\rangle$ {
    requiere: { Todos los elementos de s son positivos }
    requiere: { No hay elementos repetidos en s }
    asegura: { resultado tiene la misma cantidad de elementos que s }
    asegura: { Los elementos de resultado son la salida de aplicar el problema raizCuadrada a uno o varios elementos de la secuencia s }
}

g) **problema** raicesCuadradasCuatro (s: $seq\langle\mathbb{Z}\rangle$): $seq\langle\mathbb{R}\rangle$ {
    requiere: { Existen elementos de s que son positivos }
    requiere: { No hay elementos repetidos en s }
    asegura: { Los elementos de resultado son la salida de aplicar el problema raizCuadrada a todos los elementos de s que son positivos }
}

h) **problema** raicesCuadradasCinco (s: $seq\langle\mathbb{Z}\rangle$): $seq\langle\mathbb{R}\rangle$ {
requiere: { Todos los elementos de s son positivos }
asegura: { Cada posición de resultado, donde la posición es menor o igual a las de s, es igual a la salida de aplicar raizCuadrada al elemento que se encuentra en esa posición en s }
}

i) **problema** raicesCuadradasSeis (s: $seq\langle\mathbb{Z}\rangle$): $seq\langle\mathbb{R}\rangle$ {
requiere: { Todos los elementos de s son positivos }
asegura: { La longitud de resultado es como máximo la misma que s }
asegura: { Cada posición de resultado, donde la posición es menor o igual a las de s, es igual a la salida de aplicar raizCuadrada al elemento que se encuentra en esa posición en s }
}

Resolución:

Vamos a proponer valores de entrada que cumplan estrictamente con los `requiere` (precondiciones) y valores de salida que cumplan con todos los `asegura` (postcondiciones).

a) `duplicar`

- **Entrada:** $x = 5$ *(Cumple True)*
- **Salida:** $resultado = 10$ *(Cumple ser el doble)*

b) `raizCuadrada`

- **Entrada:** $x = 9$ *(Cumple ser positivo)*
- **Salida:** $resultado = 3{,}0$ *(Cumple ser la raíz cuadrada)*

c) `enteroMasCercanoPositivo`

- **Entrada:** $x = 3{,}8$ *(Cumple True)*
- **Salida:** $resultado = 4$ *(Cumple ser el más cercano y ser positivo)*

d) `raicesCuadradasUno`

- **Entrada:** $s = \langle 4, 9, 16\rangle$ *(Cumple ser todos positivos y sin repetidos)*
- **Salida:** $resultado = \langle 2{,}0, 3{,}0, 4{,}0\rangle$ *(Mantiene cantidad, calcula raíces y **mantiene el orden**)*

e) `raicesCuadradasDos`

- **Entrada:** $s = \langle 4, 9, 16\rangle$
- **Salida:** $resultado = \langle 3{,}0, 2{,}0, 4{,}0\rangle$ *(Es válido porque la especificación pide las raíces y la misma cantidad, pero **no exige** que se mantenga el orden original).*

f) `raicesCuadradasTres`

- **Entrada:** $s = \langle 4, 9, 16 \rangle$
- **Salida:** $resultado = \langle 2{,}0, 2{,}0, 2{,}0 \rangle$ *(Válido porque tiene 3 elementos y todos resultan de aplicar la raíz a üno o varios.*^{el}*ementos de s, en este caso elegimos siempre el 4).*

g) `raicesCuadradasCuatro`

- **Entrada:** $s = \langle -5, 4, -10, 9 \rangle$ *(Cumple que .*^{ex}*isten.*^{el}*ementos positivos y no hay repetidos)*
- **Salida:** $resultado = \langle 2{,}0, 3{,}0 \rangle$ *(Aplica la raíz sólo a los positivos de la secuencia)*

h) `raicesCuadradasCinco`

- **Entrada:** $s = \langle 4, 9 \rangle$
- **Salida:** $resultado = \langle 2{,}0, 3{,}0, 99{,}9, -1{,}5 \rangle$ *(Es válido. La postcondición restringe qué pasa en los índices menores o iguales al tamaño de s, pero* ***no restringe la longitud máxima****. Puede haber "basura.*^{a}*l final).*

i) `raicesCuadradasSeis`

- **Entrada:** $s = \langle 4, 9, 16 \rangle$
- **Salida:** $resultado = \langle 2{,}0 \rangle$ *(Válido. Pide que la longitud sea çomo máximo"la de s, así que puede ser menor. El único elemento devuelto coincide con la raíz en esa posición).*

Ejercicio 2. A partir de las especificaciones del Ejercicio 1, responder las siguientes preguntas:

1. En los problemas `raicesCuadradas` que utilizan el problema `raizCuadrada`, ¿Se puede eliminar el requiere "Todos los elementos de s son positivos¿ Justificar.
2. ¿Qué consecuencia tiene en el resultado la diferencia de asegura entre los problemas `raicesCuadradasUno` y `raicesCuadradasDos`? Buscar un ejemplo de valor de entrada donde cada problema tenga distinto valor de salida.
3. De acuerdo con la respuesta del ítem anterior, ¿un algoritmo que satisface la especificación de `raicesCuadradasUno`, también satisface la especificación de `raicesCuadradasDos`? ¿y al revés?
4. Explicar en palabras las diferencias entre los problemas `raicesCuadradasCinco` y `raicesCuadradasSeis`. ¿Cómo influye el asegura de longitud máxima? Dada la entrada $s = \langle 3, 9, 11, 15, 18 \rangle$ ¿es $\langle \sqrt{3}, \sqrt{9} \rangle$ una salida válida para ambos problemas? Y sea la entrada $s = \langle 3, 9, 11 \rangle$, ¿es $\langle \sqrt{3}, \sqrt{9}, \sqrt{11}, \sqrt{13} \rangle$ una salida válida para el problema `raicesCuadradasCinco`?
5. ¿Qué cambia en el problema `raicesCuadradasCuatro` agregar un asegura que diga que `resultado` tiene la misma longitud que `s`? Pensar ejemplos de valores de salida que cambien con este nuevo asegura.
6. Si los problemas `raicesCuadradasDos` y `raicesCuadradasTres` tienen el mismo resultado para la misma entrada (una secuencia específica de números), ¿quiere decir que son el mismo problema?
7. ¿Qué ocurre si eliminamos los requiere "no hay repetidos¿ Dada la entrada $s = \langle 4, 1, 1 \rangle$, ¿es $\langle 2, 2, 1 \rangle$ una salida válida para el problema `raicesCuadradasDos`?

Resolución:

1. **No se puede eliminar.** El problema `raizCuadrada` tiene como precondición (`requiere`) que el número ingresado sea estrictamente positivo. Si las funciones principales no exigen que todos los elementos sean positivos desde el principio, podrían intentar pasarle un número negativo a `raizCuadrada`, violando su contrato y causando un error o comportamiento indefinido en la ejecución.
2. **Diferencia:** `raicesCuadradasUno` exige que el resultado mantenga exactamente el mismo orden que la secuencia original (es más fuerte). `raicesCuadradasDos` no dice nada sobre el orden, por lo que cualquier permutación de las raíces válidas es correcta (es más débil/permisivo).
   Ejemplo: Si entrada $s = \langle 4, 9 \rangle$.
   - Para `Uno`, la *única* salida válida es $resultado = \langle 2{,}0, 3{,}0 \rangle$.
   - Para `Dos`, son válidas tanto $resultado = \langle 2{,}0, 3{,}0 \rangle$ como $resultado = \langle 3{,}0, 2{,}0 \rangle$.
3. **Uno satisface a Dos, pero no al revés.**
   Cualquier algoritmo que resuelva `Uno` devuelve los elementos en el mismo orden. Al hacerlo, automáticamente cumple con las exigencias menores de `Dos` (tener la misma cantidad y ser las raíces correspondientes).
   Al revés no: un algoritmo que resuelve `Dos` podría devolver los elementos desordenados (como en el ejemplo anterior), lo cual viola el `asegura` de orden estricto de `Uno`.

4. **Diferencia:** `raicesCuadradasCinco` exige que las primeras posiciones tengan las raíces correspondientes, pero no le importa qué hay después en la memoria (permite devolver "basuraïnfinita al final de la secuencia). `raicesCuadradasSeis` restringe esto agregando una cota de longitud máxima (como máximo la misma que s).
   - Para $s = \langle 3, 9, 11, 15, 18 \rangle$, la salida $\langle \sqrt{3}, \sqrt{9} \rangle$ **es válida para ambos**. En ambos se cumple que en los índices 0 y 1 están las raíces de s. En el `Cinco` esto basta. En el `Seis` también, porque la longitud es 2, que cumple con ser "menor o igual a 5".
   - Para $s = \langle 3, 9, 11 \rangle$, la salida $\langle \sqrt{3}, \sqrt{9}, \sqrt{11}, \sqrt{13} \rangle$ **es válida para el Cinco** (en los índices 0, 1 y 2 están las raíces correctas de s, y el índice 3 de sobra está permitido por no haber límite). Sin embargo, **no sería válida para el Seis**, porque su longitud (4) excede a la longitud original de s (3).
5. **Cambia el tratamiento de los negativos.** Originalmente, `Cuatro` filtra los positivos y devuelve sólo las raíces de estos (por lo que el resultado se .ªchica"si había negativos). Si obligamos a que el resultado tenga la misma longitud que s, el programador está forzado a rellenar"los huecos dejados por los números negativos con algún valor basura para no perder longitud.
   Ejemplo: Para $s = \langle -5, 4 \rangle$.
   - Original: $resultado = \langle 2{,}0 \rangle$ (longitud 1).
   - Con el nuevo `asegura`: Tendría que ser $resultado = \langle 99{,}9, 2{,}0 \rangle$ (longitud 2), rellenando el espacio del -5 con cualquier valor ya que no se le puede aplicar raíz.
6. **No son el mismo problema.** Dos problemas se definen por sus especificaciones teóricas (contratos), no por cómo se comportan para un caso de prueba en particular. El problema `Tres` es muchísimo más permisivo: permite que devuelvas una secuencia donde todos los elementos son la raíz del primer número repetida muchas veces. `Dos` exige aplicar la raíz a *todos* los elementos distintos. Sus conjuntos de programas que los satisfacen son diferentes.
7. **No es válida.** Si $s = \langle 4, 1, 1 \rangle$, el contrato de `raicesCuadradasDos` dice que el resultado debe tener la misma cantidad de elementos (tres) y que esos elementos son el resultado de aplicarle la raíz a *todos* los de s.
   - Aplicado al 4, da 2.
   - Aplicado al primer 1, da 1.
   - Aplicado al segundo 1, da 1.

   Por lo tanto, la "bolsa"de elementos del resultado debe tener exactamente un 2 y dos 1s. La salida propuesta $\langle 2, 2, 1 \rangle$ contiene dos 2s y un 1, por lo que es incorrecta y no satisface la especificación.

Ejercicio 3. Dada la siguiente especificación del problema de ordenar una secuencia de enteros, en la que se debe tomar una secuencia de números enteros y devolver los mismos elementos ordenados de menor a mayor:

```
problema ordenar (s: seq<Z>) : seq<Z> {
    requiere: {True}
    asegura: {resultado es una secuencia en la cual cada elemento
              es estrictamente mayor que el anterior}
}
```

responder la siguientes preguntas:

a) Dadas $s = \langle 4, 3, 5 \rangle$ como secuencia de entrada, ¿es $resultado = \langle 3, 4, 5 \rangle$ una solución válida según la especificación?

b) Dados $s = \langle 4, 3, 3, 5 \rangle$ como secuencia de entrada, ¿es $resultado = \langle 3, 3, 4, 5 \rangle$ una solución válida según la especificación? Corregir la especificación modificando el requiere.

c) Si tomamos $s = \langle 4, 3, 5 \rangle$ como secuencia de entrada, ¿es $resultado = \langle 3, 4 \rangle$ una solución válida según la especificación? Corregir la especificación modificando el asegura.

d) Si tomamos $s = \langle 4, 3, 5 \rangle$ como secuencia de entrada, ¿es $resultado = \langle 3, 4, 5, 6 \rangle$ una solución válida según la especificación? Corregir la especificación modificando el asegura.

e) Dadas $s = \langle 8, 5, 7 \rangle$ como secuencia de entrada, ¿es $resultado = \langle 1, 2, 3 \rangle$ una solución válida según la especificación?

f) Escribir una especificación que permita recibir cualquier secuencia de enteros s como parámetro y garantice que resultado contiene el resultado de ordenar correctamente los elementos s de menor a mayor.

Resolución:

a) Sí, es válida. En $\langle 3, 4, 5 \rangle$, cada elemento es estrictamente mayor que el anterior ($3 < 4 < 5$). Cumple perfectamente el único `asegura` de la especificación.

b) No, no es válida. En $\langle 3, 3, 4, 5 \rangle$, el segundo 3 no es .^estrictamente mayor"que el primer 3 ($3 \not> 3$). Por lo tanto, rompe el `asegura`.
Corrección: Como el problema nos pide modificar el `requiere` para arreglar esto, debemos prohibir que nos pasen secuencias con números repetidos (ya que el `asegura` actual no los soporta).

```
requiere: {s no tiene elementos repetidos}
```

c) Sí, es válida. En la secuencia $\langle 3, 4 \rangle$, el 4 es estrictamente mayor que el 3. El `asegura` actual nunca nos obligó a devolver la misma cantidad de elementos ni a que sean los mismos de la entrada, por lo que devolver una secuencia más corta (perdiendo el 5) cumple el contrato.
Corrección: Debemos agregar un `asegura` para garantizar que no se pierdan elementos.

```
asegura: {Todos los elementos de s pertenecen a resultado}
asegura: {resultado tiene la misma longitud que s}
```

(Nota: con estas dos juntas ya evitamos que se çoman.^elementos).

d) **Sí, es válida.** En $\langle 3, 4, 5, 6\rangle$, se cumple que $3 < 4 < 5 < 6$. Al igual que en el inciso anterior, la especificación actual es tan débil que no prohíbe inventar números nuevos.
Corrección: Al igual que en el inciso c, debemos forzar a que la salida tenga los mismos elementos que la entrada.

```
asegura: {Todos los elementos de resultado pertenecen a s}
asegura: {resultado tiene la misma longitud que s}
```

e) **Sí, es válida.** ¡Este es el ejemplo más extremo de que la especificación actual está rota! La salida $\langle 1, 2, 3\rangle$ está estrictamente ordenada, por lo que cumple el contrato. Al `asegura` actual no le importa en lo más mínimo qué te pasaron en s, por lo que un algoritmo que siempre devuelva $\langle 1, 2, 3\rangle$ sin importar la entrada sería considerado çorrecto"según este contrato.

f) **Especificación final corregida:** Para arreglar todos los problemas anteriores de una vez, tenemos que permitir repetidos cambiando el .^es^trictamente mayor"por un "menor o igual", y asegurar que la salida es exactamente una permutación de la entrada.

```
problema ordenar (s: seq<Z>) : seq<Z> {
    requiere: {True}
    asegura: {resultado tiene exactamente los mismos elementos que s,
              con la misma cantidad de apariciones (es una permutación de s)}
    asegura: {Cada elemento de resultado es menor o igual al siguiente}
}
```

Ejercicio 4. Se desea especificar el problema de reemplazar cada elemento de una secuencia de enteros por su doble y se cuenta con la siguiente especificación:

```
problema duplicarTodos (s: seq<Z>) : seq<Z> {
    requiere: {True}
    asegura: {resultado tiene la misma cantidad de elementos que s}
}
```

a) ¿Qué problemas tiene la especificación dada? Dar ejemplos de valores para resultado que satisfagan la especificación pero no sean respuestas correctas.

b) Indicar cuál/es de los siguientes asegura debería/n ser agregado/s a la especificación. Justificar en cada caso por qué deberían o no ser agregados.

- asegura: {Para cada valor z que pertenece a s, hay algún valor en resultado que es la salida de duplicar(z)}
- asegura: {En cada posición de resultado, el valor es mayor al valor en esa misma posición de s}
- asegura: {En cada posición de resultado, el valor es igual a la salida de aplicar duplicar al valor en esa misma posición de s}
- asegura: {Todos los elementos de resultado son números pares}

Nota: el problema duplicar() está especificado en el Ejercicio 1.

Resolución:

a) **Problemas de la especificación:**
El único contrato que exige esta especificación es sobre la **longitud** de la secuencia de salida, pero no restringe en absoluto el **contenido** de la misma. Un programador podría devolver una secuencia llena de ceros (o de cualquier otro número) y, mientras tenga la misma longitud que `s`, el contrato se considerará cumplido, aunque no hayamos duplicado nada.
Ejemplo:

- Entrada: $s = \langle 1, 2, 3 \rangle$
- Salida esperada (correcta): $resultado = \langle 2, 4, 6 \rangle$
- Salida válida por la especificación (incorrecta en la vida real): $resultado = \langle 0, 0, 0 \rangle$ (o incluso devolver la misma secuencia original $\langle 1, 2, 3 \rangle$).

b) **Análisis de los posibles `asegura`:**

- **asegura: {Para cada valor z que pertenece a s, hay algún valor en resultado que es la salida de duplicar(z)}**
NO se debe agregar (por sí solo). Aunque es una propiedad verdadera, es "débil" porque no exige que se respeten las posiciones ni las cantidades exactas (cardinalidad). Por ejemplo, si $s = \langle 2, 2 \rangle$, un resultado válido según este asegura podría ser $resultado = \langle 4, 99 \rangle$. Para el 2 de s, existe un 4 en `resultado`, pero el 99 es basura inventada.

- **asegura: {En cada posición de resultado, el valor es mayor al valor en esa misma posición de s}**
NO se debe agregar. Esta afirmación es matemáticamente falsa para números negativos y para el cero. Por ejemplo, si un elemento de `s` es 0, su doble es 0, y $0 \not> 0$. Si el elemento es -2, su doble es -4, y $-4 \not> -2$. Rompería el programa para cualquier número ≤ 0.

- **asegura: {En cada posición de resultado, el valor es igual a la salida de aplicar duplicar al valor en esa misma posición de s}**
SÍ se debe agregar. Esta es la postcondición perfecta. Ata el comportamiento posición a posición (mapeo 1 a 1). Garantiza que en el índice i del `resultado` esté exactamente el doble del elemento que está en el índice i de `s`. Esto, sumado al asegura original de la longitud, define el problema sin ambigüedades.

- **asegura: {Todos los elementos de resultado son números pares}**
NO se debe agregar. Es una propiedad matemática cierta (el doble de cualquier entero es par), pero no define el problema de la duplicación de `s`. Si $s = \langle 1 \rangle$, un resultado como $resultado = \langle 8 \rangle$ cumple con ser par y tener la misma cantidad de elementos, pero no es el doble de 1.

Ejercicio 5. (Frecuencia de bondis) A Ciudad Universitaria (CU) llegan 8 líneas de colectivos: 28, 33, 34, 37, 45, 107, 160 y 166. Con el fin de controlar la frecuencia diaria de cada línea, un grupo de investigación del Departamento de Computación instaló cámaras y un sistema de reconocimiento de imágenes en el ingreso al predio. Durante el día, el sistema identifica el número de línea de cada colectivo que ingresa y lo registra en una secuencia ordenada.

a) Especificar el problema `cantidadColectivosDeLinea` que a partir de una secuencia de colectivos registrada por el sistema de reconocimiento y el número de una línea que llega a CU, devuelva cuántos colectivos de esa línea ingresaron durante el día.

b) Especificar el problema `lineaConMejorFrecuencia` que, a partir de dos números de líneas y una secuencia registrada por el sistema, devuelva cuál de las dos líneas tiene mejor frecuencia diaria. Sugerencia: utilizar `cantidadColectivosDeLinea`.

Resolución:

Para estas especificaciones, vamos a definir como precondición lógica (`requiere`) que los colectivos evaluados pertenezcan estrictamente al conjunto de líneas válidas que entran a Ciudad Universitaria.

a) Especificación de `cantidadColectivosDeLinea`

```
problema cantidadColectivosDeLinea (s: seq<Z>, linea: Z) : Z {
    requiere: {linea pertenece al conjunto {28, 33, 34, 37, 45, 107, 160, 166}}
    requiere: {Todos los elementos de la secuencia s pertenecen al conjunto
               {28, 33, 34, 37, 45, 107, 160, 166}}
    asegura: {resultado es la cantidad de veces que aparece el valor de
              la variable linea en la secuencia s}
}
```

b) Especificación de `lineaConMejorFrecuencia`

Entendemos por "mejor frecuencia diaria.ª la línea que ingresó mayor cantidad de veces al predio. Como nos sugieren usar la función anterior, la invocamos directamente dentro de nuestras postcondiciones (`asegura`).

```
problema lineaConMejorFrecuencia (s: seq<Z>, linea1: Z, linea2: Z) : Z {
    requiere: {linea1 y linea2 pertenecen al conjunto {28, 33, 34, 37, 45, 107, 160, 166}}
    requiere: {Todos los elementos de la secuencia s pertenecen al conjunto
               {28, 33, 34, 37, 45, 107, 160, 166}}
    asegura: {resultado es igual a linea1 o resultado es igual a linea2}
    asegura: {Si cantidadColectivosDeLinea(s, linea1) >= cantidadColectivosDeLinea(s, linea2
              entonces resultado es igual a linea1}
    asegura: {Si cantidadColectivosDeLinea(s, linea2) > cantidadColectivosDeLinea(s, linea1)
              entonces resultado es igual a linea2}
}
```

Nota sobre el empate: Fijate que en el segundo asegura le pusimos un "Mayor o Igual"(`>=`). Esto es una decisión de diseño para que, en caso de que ambas líneas hayan pasado exactamente la misma cantidad de veces, el programa no se rompa y simplemente devuelva `linea1` para desempatar.

Ejercicio 6. (Control de Calificaciones en el Departamento de Ciencias)

En el prestigioso Departamento de Ciencias de una reconocida universidad, un grupo de estudiantes se ha embarcado en su jornada académica, cursando diversas materias bajo el cuidadoso seguimiento del cuerpo docente. En este departamento, cada estudiante ha sido registrado con su respectivo nombre y apellido, asegurándose de que no existan duplicados.

La información relevante de las cursadas de los estudiantes se encuentra almacenada en un sistema que contiene una secuencia de tuplas en formato `<Materia, Calificación obtenida>`. Las calificaciones se encuentran en un rango numérico entre 0 y 10.

El Departamento ha establecido una política de aprobación y recursado que dicta que si un estudiante aprueba una materia con una calificación igual o superior a 4, no deberá volver a cursarla, quedando esta materia registrada como aprobada en su expediente académico. Sin embargo, si no logra alcanzar la calificación mínima de aprobación, tendrá la posibilidad de recursar la materia en un futuro intento.

Además, existe en el sistema una estructura de datos llamada `CalificacionesDelDC` que contiene la información de los estudiantes y las calificaciones en sus cursadas. Esta estructura es una secuencia de tuplas en el formato `<Alumno, Cursada>`, donde Alumno es el nombre y apellido del estudiante y Cursada es la secuencia de tuplas mencionada previamente.

Considerando esta información y los siguientes renombres de tipos:

- `Renombre Alumno = String`
- `Renombre Materia = String`
- `Renombre Cursada = seq<Materia, R>`
- `Renombre CalificacionesDelDC = seq<Alumno, Cursada>`

a) Especificar problema `promedioDeAlumno (alumno: Alumno, calificaciones: CalificacionesDelDC)`
`R`

b) Especificar el problema que, dado el listado de materias cursadas por un estudiante, indique en qué materia tuvo mayor calificación. ¿Cómo se debe modificar la especificación para devolver el listado de materias en las cuales tuvo mejor calificación?

c) Especificar el problema que, dada una materia y las calificaciones del DC, devuelve todos los estudiantes que cursaron y aprobaron esa materia. ¿Cómo debe modificarse la especificación para que los nombres se devuelvan en orden alfabético?, ¿este cambio reduce o amplía la cantidad de programas que resolverían el problema?

d) Especificar el problema de devolver una secuencia con los promedios de todos los estudiantes.

Resolución:

a) Especificación de `promedioDeAlumno`

```
problema promedioDeAlumno (alumno: Alumno, calificaciones: CalificacionesDelDC) : R {
    requiere: {El alumno pertenece a la secuencia calificaciones}
    requiere: {La Cursada del alumno tiene al menos una materia
               (para evitar división por cero)}
    asegura: {resultado es igual a la suma de todas las calificaciones en la
              Cursada del alumno dividida por la cantidad de materias en esa Cursada}
}
```

b) Especificación de mayor calificación y su modificación

1. Especificación para una única materia:

```
problema materiaConMayorCalificacion (cursada: Cursada) : Materia {
    requiere: {cursada tiene al menos una materia}
    asegura: {La materia resultado pertenece a cursada}
    asegura: {No existe ninguna otra materia en cursada con una calificación
              estrictamente mayor a la obtenida en la materia resultado}
}
```

2. Modificación para devolver un listado (puede haber empates en la nota máxima):
Cambiamos el tipo de retorno a `seq<Materia>` y ajustamos los asegura para garantizar que estén **todas** las que tienen nota máxima y **ninguna** que no la tenga.

```
problema materiasConMejorCalificacion (cursada: Cursada) : seq<Materia> {
    requiere: {cursada tiene al menos una materia}
    asegura: {Todas las materias en resultado pertenecen a cursada}
    asegura: {No hay elementos repetidos en resultado}
    asegura: {Todas las materias de cursada que tienen la calificación máxima
              pertenecen a resultado}
    asegura: {Ninguna materia en resultado tiene una calificación menor a la
              calificación máxima de la cursada}
}
```

c) Especificación de alumnos aprobados y orden alfabético

1. Especificación base:

```
problema aprobaronMateria (materia: Materia,
                           calificaciones: CalificacionesDelDC) : seq<Alumno> {
    requiere: {True}
    asegura: {No hay alumnos repetidos en resultado}
    asegura: {Todos los alumnos en resultado pertenecen a calificaciones y
              tienen la materia en su Cursada con una nota mayor o igual a 4}
    asegura: {Todos los alumnos de calificaciones que tienen la materia en su
              Cursada con nota mayor o igual a 4, pertenecen a resultado}
}
```

2. Modificación para orden alfabético y análisis:
Para que devuelva ordenado, simplemente hay que agregar un asegura extra:

```
    asegura: {Los alumnos en resultado están ordenados alfabéticamente}
```

Análisis: Este cambio **reduce** la cantidad de programas que resolverían el problema. Al agregar una postcondición más restrictiva, los programas que antes eran válidos (los que devolvían la lista desordenada) ahora dejan de serlo. Solo sobreviven los algoritmos que, además de filtrar, ordenan. El conjunto de programas válidos se achica.

d) Especificación de secuencia de promedios

Podemos aprovechar la función del inciso **a** para hacer la postcondición mucho más legible.

```
problema promediosDeTodos (calificaciones: CalificacionesDelDC) : seq<R> {
    requiere: {Todos los alumnos en calificaciones tienen al menos una materia
               en su Cursada}
    asegura: {resultado tiene la misma longitud que calificaciones}
    asegura: {Para cada índice i válido de calificaciones, el valor en el índice i
              de resultado es igual a promedioDeAlumno del alumno que está en
              el índice i de calificaciones}
}
```

---

2. Principio de sustitución

Ejercicio 7. Contamos con las siguientes especificaciones del problema `pares`:

```
problema pares1 (s: seq<Z>) : seq<Z> {
    requiere: {s no tiene elementos repetidos}
    asegura: {Los elementos de resultado son pares y pertenecen a s}
    asegura: {Los elementos de s que son pares, pertenecen a resultado}
    asegura: {resultado no tiene elementos repetidos}
}

problema pares2 (s: seq<Z>) : seq<Z> {
    requiere: {s no tiene elementos repetidos}
    asegura: {Los elementos de resultado son pares y pertenecen a s}
    asegura: {Los elementos de s que son pares, pertenecen a resultado}
    asegura: {resultado no tiene elementos repetidos}
    asegura: {resultado está ordenada de manera creciente}
}
```

a) Si contamos con un algoritmo P que satisface `pares1`, ¿satisface P la especificación `pares2`? Justificar.

b) Si contamos con un algoritmo P que satisface `pares2`, ¿satisface P la especificación `pares1`? Justificar.

c) ¿Cuál es la relación de fuerza entre la postcondición de `pares1` y la de `pares2`?

Resolución:

a) No necesariamente. Un algoritmo P que satisface `pares1` cumple con devolver los números pares sin repetidos, pero la especificación `pares1` no le exige ningún orden en particular. Por lo tanto, P podría devolver la secuencia desordenada. Si usamos este algoritmo P pretendiendo que cumpla `pares2`, fallaría al violar el último asegura (el que exige que el resultado esté ordenado de manera creciente).
Ejemplo: Si $s = \langle 8, 3, 4 \rangle$, P (`pares1`) podría devolver $\langle 8, 4 \rangle$. Esto es inválido para `pares2`, que exige $\langle 4, 8 \rangle$.

b) **Sí, totalmente.** Un algoritmo P que satisface `pares2` hace todo lo que pide `pares1` y además los entrega ordenados. Como `pares1` no prohíbe que los devuelvas ordenados (simplemente no le importa), un resultado ordenado sigue siendo perfectamente válido. Por el principio de sustitución, si alguien espera que se cumpla `pares1`, podés darle un algoritmo que cumple `pares2` y nunca se va a dar cuenta porque el contrato original se respeta a rajatabla.

c) **Relación de fuerza:** La postcondición de `pares2` es **más fuerte** que la postcondición de `pares1`.

Justificación: Recordemos de la Guía 1 que una proposición A es más fuerte que B si $A \rightarrow B$ es una tautología. En `pares2` tenemos todas las mismas exigencias que en `pares1` más una exigencia adicional (estar ordenado). Podemos pensarlo como una gran conjunción:

$$Asegura_{pares2} \equiv Asegura_{pares1} \wedge \text{“‘está ordenado”}$$

Es evidente lógicamente que $(Asegura_{pares1} \wedge \text{“‘está ordenado”}) \rightarrow Asegura_{pares1}$. Por lo tanto, quien cumple la postcondición 2 cumple obligatoriamente la 1, haciendo a la 2 más estricta (más fuerte).

Ejercicio 8. Contamos con las siguientes especificaciones del problema `sumarAbsMayorA5`:

```
problema sumarAbsMayorA5-version1 (s: seq<Z>) : Z {
    requiere: {True}
    asegura: {resultado es la sumatoria de todos los elementos de s
              cuyo valor absoluto es mayor a 5}
}

problema sumarAbsMayorA5-version2 (s: seq<Z>) : Z {
    requiere: {Todos los elementos de s son positivos}
    asegura: {resultado es la sumatoria de todos los elementos de s
              cuyo valor absoluto es mayor a 5}
}

problema sumarAbsMayorA5-version3 (s: seq<Z>) : Z {
    requiere: {Todos los elementos de s son mayores a 10}
    asegura: {resultado es la sumatoria de todos los elementos de s
              cuyo valor absoluto es mayor a 5}
}
```

a) ¿Cuál es la relación de fuerza entre los requiere de cada especificación?

b) ¿Cuál de las especificaciones tiene el dominio más restringido y cuál menos?

c) Desde el punto de vista de un programador, ¿qué especificación es más fácil de implementar? Justificar.

d) Desde el punto de vista de un usuario, ¿qué contrato es más conveniente? Justificar.

Resolución:

a) **Relación de fuerza de los `requiere`:**

- `Req1`: `True` (es la proposición más débil posible).
- `Req2`: $x > 0$ para todo $x \in s$.
- `Req3`: $x > 10$ para todo $x \in s$.

Si un número es mayor a 10, obligatoriamente es mayor a 0. Y si es mayor a 0, obligatoriamente hace que `True` sea verdadero. Por implicación lógica (`Req3` $\rightarrow$ `Req2` $\rightarrow$ `Req1`):

`Req3` es el más fuerte, luego `Req2`, y `Req1` es el más débil.

b) **Dominio:**

- **Más restringido:** La **versión 3**. Su `requiere` (fuerte) recorta la entrada a un subconjunto muy pequeño (secuencias donde todos los números sean estrictamente mayores a 10).
- **Menos restringido:** La **versión 1**. Su `requiere` (débil) no filtra nada, acepta el dominio universal (cualquier secuencia de enteros).

c) **Para el programador:** La más fácil de implementar es la **versión 3**.

Justificación: Un contrato fuerte (`requiere` estricto) le quita responsabilidad al programador. Al garantizarle que todos los elementos de la lista de entrada ya son mayores a 10, el programador sabe por matemática básica que sus valores absolutos SIEMPRE serán mayores a 5. Por lo tanto, no necesita programar estructuras condicionales (un `if`) ni calcular el valor absoluto (`abs()`). El código se reduce simplemente a un ciclo que suma todos los elementos de la lista de corrido, sin preguntar nada.

d) **Para el usuario:** El contrato más conveniente es la **versión 1**.

Justificación: El usuario quiere la herramienta más versátil posible. Al tener un `requiere` débil (`True`), el usuario se ahorra el trabajo de tener que pre-procesar, filtrar o limpiar la lista antes de llamar a la función. Le tira cualquier secuencia (con negativos, ceros, etc.) y la función resuelve el problema haciéndose cargo internamente de evaluar los valores absolutos.